

PARACASYA_REGLAS_CODEX.md

Reglas de trabajo para ChatGPT/Codex

Proyecto: ParacasYa Market Online

1. Propósito del documento

Este documento define las reglas obligatorias para trabajar con el proyecto **ParacasYa Market Online** usando ChatGPT, Codex o cualquier asistente de programación.

El objetivo es evitar errores, proteger secretos, no romper funcionalidades ya estables y trabajar siempre por fases controladas.

ParacasYa Market ya funciona como un MVP avanzado con:

- Landing pública.
- Tienda `/cliente`.
- Caja `/caja`.
- Admin `/admin`.
- Supabase.
- Edge Functions.
- Telegram.
- WhatsApp.
- QR por hotel/Airbnb.
- Seguimiento visual del pedido.
- Catálogo real de productos.
- Catálogo de cervezas aplicado.
- Build aprobado.

Por eso, cualquier cambio debe hacerse con cuidado.

2. Estado técnico base

Carpeta local del proyecto

C:\Users\Usuario\Documents\ParacasYa Market Online

Rama principal

```
main
```

Último commit documentado

```
90ea544 feat: agregar catalogo de cervezas
```

Estado base esperado

```
git status --short
```

Debe estar limpio antes de iniciar una fase, salvo que Walter indique lo contrario.

3. Regla principal

No modificar nada fuera del alcance de la fase solicitada.

Cada tarea debe tener:

1. Objetivo claro.
 2. Archivos esperados a tocar.
 3. Restricciones.
 4. Validación.
 5. Resumen final.
 6. Confirmación de que no se tocaron áreas sensibles.
-

4. Zonas sensibles del proyecto

Estas zonas no se deben tocar salvo autorización explícita de Walter:

- `.env`
- `.env.local`
- Secrets
- Variables de Vercel
- Variables de Supabase Functions
- RLS
- Políticas de escritura
- Service role
- Edge Functions

- Tabla `orders`
 - Tabla `order_items`
 - Lógica de creación de pedidos
 - Lógica de caja
 - WhatsApp
 - Telegram
 - QR/origen hotel
 - Seguimiento de pedido
-

5. Reglas sobre secretos

Nunca imprimir, copiar, mostrar ni confirmar valores reales de:

- `VITE_SUPABASE_URL`
- `VITE_SUPABASE_ANON_KEY`
- `VITE_INTERNAL_NOTIFY_TOKEN`
- `VITE_INTERNAL_PIN`
- `ADMIN_CATALOG_TOKEN`
- `INTERNAL_NOTIFY_TOKEN`
- `TELEGRAM_BOT_TOKEN`
- `TELEGRAM_CHAT_ID`
- `SUPABASE_SERVICE_ROLE_KEY`
- `SERVICE_ROLE_KEY`
- Cualquier token, API key, password o secret.

Se pueden mencionar nombres de variables, pero no sus valores.

Correcto:

`VITE_SUPABASE_URL` está configurada.

Incorrecto:

`VITE_SUPABASE_URL=https://...`

6. Reglas sobre `.env` y variables

No modificar:

```
.env  
.env.local
```

No crear nuevos archivos `.env` sin autorización.

No cambiar variables de Vercel.

No cambiar variables de Supabase Edge Functions.

Si una fase necesita una variable nueva, primero explicar:

- Nombre sugerido.
- Dónde se usaría.
- Si va en local, Vercel o Supabase.
- Nivel de sensibilidad.
- Riesgo de exponerla.

Esperar confirmación antes de implementarla.

7. Reglas sobre Git

Antes de iniciar cualquier fase:

```
git status --short  
git log --oneline -3
```

Si hay cambios pendientes no relacionados, detenerse y avisar.

Durante la fase:

- No hacer commit salvo que Walter lo pida.
- No hacer push salvo que Walter lo pida.
- No mezclar varias fases en un mismo cambio.
- No incluir archivos sensibles.
- No subir `.env`, `.env.local`, tokens ni credenciales.

Al finalizar:

```
git status --short  
git diff  
npm run build
```

Solo hacer commit/push cuando Walter diga explícitamente:

```
hacé commit y push
```

o

```
subilo a GitHub
```

8. Reglas sobre build

Toda fase debe terminar con:

```
npm run build
```

Si el build pasa con warning de chunk grande, no es bloqueante.

Warning conocido:

```
Some chunks are larger than 500 kB after minification.
```

Si el build falla:

1. No hacer commit.
 2. No hacer push.
 3. Explicar el error.
 4. Indicar archivo/línea si aparece.
 5. Proponer arreglo.
 6. Esperar confirmación si el arreglo excede la fase.
-

9. Reglas sobre Supabase

El proyecto usa Supabase para:

- Productos.
- Categorías.
- Pedidos.
- Items de pedido.
- Tienda abierta/cerrada.
- QR/origen por hotel registrado en pedidos.

Tablas principales:

- products
- categories
- orders
- order_items
- store_settings

No tocar Supabase remoto salvo que la fase lo pida explícitamente.

No modificar RLS sin autorización.

No abrir escrituras anónimas.

No agregar políticas permisivas.

No usar service role en frontend.

No crear DELETE.

No borrar datos reales.

10. Reglas sobre SQL

Si se necesita agregar productos, categorías o configuración:

1. Crear archivo SQL local en `supabase/`.
2. Debe ser idempotente.
3. No debe duplicar registros.
4. No debe borrar datos.
5. No debe tocar RLS salvo que se pida.
6. No debe abrir escrituras públicas.
7. No debe usar datos sensibles.
8. Debe explicar si se aplicó remoto o quedó pendiente.

Ejemplo de archivo correcto:

```
supabase/paracasya_beer_catalog.sql
```

Ese archivo agregó cervezas de forma segura e idempotente.

11. Reglas sobre productos

Los productos reales se cargan desde Supabase.

Servicio relacionado:

```
src/services/productsService.js
```

Reglas:

- No inventar productos en frontend si deben estar en base de datos.
 - No reemplazar catálogo existente sin autorización.
 - No borrar productos.
 - Para ocultar productos, usar `is_available = false`.
 - Para destacar productos, usar `is_featured = true`.
 - Si no hay imagen, usar `image_url = null`.
 - Si se agregan productos por SQL, evitar duplicados por nombre/slug según esquema.
-

12. Reglas sobre categorías

Las categorías se cargan desde Supabase.

Servicio relacionado:

```
src/services/categoriesService.js
```

Reglas:

- No duplicar categorías similares.
 - Si existe `Alcohol`, no crear innecesariamente `Bebidas alcohólicas`.
 - Si existe `Bebidas alcohólicas`, reutilizarla.
 - Las categorías públicas deben usar `is_active = true`.
 - No romper los filtros de `/cliente`.
-

13. Reglas sobre alcohol

Los productos alcohólicos deben incluir siempre aviso responsable.

Texto recomendado:

Venta solo para mayores de 18 años. Tomar bebidas alcohólicas en exceso es dañino.

O versión operativa:

Venta solo para mayores de 18 años. Podemos solicitar confirmación de edad al entregar.

No implementar venta de alcohol sin aviso visible.

No presentar alcohol como producto para menores.

No hacer promociones que incentiven consumo irresponsable.

14. Reglas sobre Cloudinary

Estado actual:

Cloudinary no está integrado como subida automática.

Actualmente:

- `/admin` permite pegar manualmente una URL de imagen.
- Esa URL se guarda en `products.image_url`.
- Puede ser una URL de Cloudinary.
- Pero la app no sube archivos automáticamente a Cloudinary.

No asumir que ya existe upload.

No agregar secretos Cloudinary al frontend.

Si se implementa upload futuro:

- Validar tipo de archivo.
 - Validar tamaño.
 - Usar preset seguro o Edge Function.
 - No exponer API secret.
 - Mantener fallback visual si la imagen falla.
-

15. Reglas sobre `/cliente`

Ruta:

`/cliente`

Archivo principal:

`src/pages/ClientePage.jsx`

Funciones actuales:

- Ver catálogo.
- Buscar productos.
- Filtrar categorías.
- Agregar al carrito.
- Checkout.
- Recibo premium.
- Origen QR/hotel.
- Seguimiento dentro del recibo.

Reglas:

- No romper carrito.
- No romper checkout.
- No romper origen QR.
- No romper tienda abierta/cerrada.
- No cambiar `createOrder` salvo que la fase lo pida.
- No inventar productos desde frontend.
- Mantener mobile-first.
- Si se hacen cambios visuales grandes, primero pedir referencia aprobada por Walter.

Importante:

Walter rechazó una propuesta visual anterior. No insistir con rediseños grandes sin dirección estética clara.

16. Reglas sobre `/caja`

Ruta:

`/caja`

Archivo principal:

```
src/pages/CajaPage.jsx
```

Funciones actuales:

- Ver pedidos reales.
- Cambiar estados.
- Ver métricas básicas.
- Ver origen QR.
- Ver comentarios.
- Abrir WhatsApp.

Reglas:

- No romper listado de pedidos.
- No romper cambio de estado.
- No romper WhatsApp.
- No modificar estructura de `orders` sin autorización.
- No agregar Realtime si la fase no lo pide.
- No agregar sonido si la fase no lo pide.

Limitación actual:

- No hay Supabase Realtime.
- No hay polling automático.
- No hay sonido de pedido nuevo.

Estas mejoras deben hacerse en una fase separada.

17. Reglas sobre `/admin`

Ruta:

```
/admin
```

Archivo principal:

```
src/pages/AdminPage.jsx
```

Funciones actuales:

- Crear productos.

- Editar productos.
- Cambiar precios.
- Pegar `image_url`.
- Activar/ocultar productos.
- Destacar productos.
- Crear/editar categorías.
- Activar/desactivar categorías.
- Tienda abierta/cerrada.
- Generador QR.

Reglas:

- No escribir catálogo con anon directo.
- Usar Edge Function `admin-catalog`.
- No exponer `ADMIN_CATALOG_TOKEN`.
- No reemplazar PIN por Auth sin fase específica.
- No tocar tienda abierta/cerrada si no se pide.
- No romper generación QR.

18. Reglas sobre Edge Functions

Funciones existentes:

```
admin-catalog
notify-order-telegram
```

No modificar Edge Functions salvo que la fase lo pida explícitamente.

`admin-catalog`

Sirve para escrituras seguras del catálogo/admin.

Variables sensibles:

- `ADMIN_CATALOG_TOKEN`
- `SUPABASE_SERVICE_ROLE_KEY`
- `SERVICE_ROLE_KEY`

Nunca exponerlas en frontend.

`notify-order-telegram`

Sirve para enviar alerta interna por Telegram.

Variables sensibles:

- `INTERNAL_NOTIFY_TOKEN`
- `TELEGRAM_BOT_TOKEN`
- `TELEGRAM_CHAT_ID`

Si Telegram falla, el pedido debe seguir creándose.

No cambiar esta lógica salvo que la fase lo pida.

19. Reglas sobre Telegram

Telegram ya funciona.

Flujo:

- Se crea pedido.
- Se crean `order_items`.
- Se llama Edge Function `notify-order-telegram`.
- Si Telegram falla, el pedido no falla.

Reglas:

- No hacer que Telegram bloquee pedidos.
 - No imprimir tokens.
 - No tocar el header `x-internal-notify-token` sin autorización.
 - No mover la lógica sin fase específica.
 - No romper el pedido por error de notificación.
-

20. Reglas sobre WhatsApp

Archivo principal:

```
src/utils/whatsapp.js
```

Reglas:

- Mantener uso de `wa.me`.
- Mantener normalización de teléfonos peruanos.
- No romper botón "Confirmar por WhatsApp".
- Si el número no es válido, debe seguir mostrando "WhatsApp no válido".
- No automatizar mensajes masivos.
- No enviar mensajes sin acción del operador.

21. Reglas sobre QR/origen hotel

Archivos relacionados:

- `src/utils/orderOrigin.js`
- `src/utils/partnerQrData.js`
- `src/components/PartnerQrCard.jsx`
- `src/components/PartnerQrCreator.jsx`

Formato:

```
/cliente?origen=hotel-paracas
```

Reglas:

- No romper query param `origen`.
- No romper guardado en `localStorage`.
- No romper envío a `orders`.
- No romper visualización en `/caja`.
- No crear tabla nueva de QR sin fase específica.
- No cambiar nombres de campos `order_origin` ni `order_origin_label` sin migración controlada.

22. Reglas sobre seguimiento de pedido

Componente:

```
src/components/OrderSuccessReceipt.jsx
```

Servicio:

```
getOrderStatus({ orderCode, orderId })
```

Reglas:

- No crear página separada de seguimiento sin fase específica.
- No romper polling cada 8 segundos.
- No romper estados visuales.
- No cambiar mapeo de estados sin validar `/caja`.
- No eliminar recuperación de recibo desde `localStorage`.

23. Reglas sobre diseño visual

El proyecto debe sentirse:

- Moderno.
- Turístico.
- Local.
- Rápido.
- Confiable.
- Fresco.
- Profesional.

No debe parecer:

- Una app genérica de hamburguesas.
- Una tienda improvisada.
- Un diseño demasiado cargado.
- Un marketplace gigante.
- Una web corporativa fría.

Regla especial:

Antes de rediseñar landing, `/cliente`, `/admin` o `/caja`, pedir confirmación estética a Walter.

Usar referencias visuales aprobadas por Walter.

24. Reglas sobre roadmap

Trabajar por fases.

Orden recomendado:

1. Prueba operativa real de producción.
2. Catálogo nocturno completo.
3. Combos comerciales.
4. Imágenes principales.
5. Aviso de alcohol visible.
6. Mejoras de caja.
7. Reportes simples.
8. Zonas de delivery.
9. Seguridad real con login.
10. Landing principal cuando haya estilo aprobado.

No saltar a fases grandes sin validar lo básico.

25. Plantilla obligatoria para prompts de Codex

Cada prompt debe seguir esta estructura:

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

[NOMBRE DE LA FASE]

Objetivo:

[OBJETIVO CONCRETO]

Contexto actual:

[QUÉ YA FUNCIONA]

Archivos esperados:

[ARCHIVOS QUE PUEDE REVISAR/MODIFICAR]

Restricciones:

- No tocar .env ni .env.local.
- No imprimir secrets.
- No tocar RLS.
- No tocar Edge Functions salvo que se pida.
- No tocar pedidos/caja/WhatsApp/Telegram salvo que se pida.
- No hacer commit ni push todavía.

Validación:

1. git status --short
2. npm run build
3. git diff
4. prueba manual indicada

Entrega final:

- archivos modificados,
- qué cambió,
- build,
- riesgos,
- áreas no tocadas,
- si recomienda commit o no.

26. Plantilla de cierre de fase

Codex debe cerrar cada fase con este formato:

```
Fase completada: [nombre]

Archivos modificados:
- ...

Qué se hizo:
- ...

Validación:
- npm run build: pasó/falló
- git status --short: ...

No se tocó:
- .env
- .env.local
- secrets
- RLS
- Edge Functions
- pedidos
- caja
- WhatsApp
- Telegram
- QR/origen hotel
- seguimiento

Pendientes:
- ...

Recomendación:
- commit/no commit
```

27. Qué hacer si algo falla

Si algo falla:

1. Detenerse.
2. No hacer commit.
3. No hacer push.
4. Explicar error exacto.
5. Indicar archivo/línea si existe.

6. Proponer solución mínima.
7. Esperar confirmación de Walter si el cambio es sensible.

Nunca arreglar un error tocando áreas sensibles sin autorización.

28. Qué hacer antes de migrar a nueva cuenta

Antes de trabajar desde el nuevo ChatGPT:

1. Subir o pegar Documento Maestro.
 2. Subir o pegar Resumen Técnico Real.
 3. Subir o pegar este documento de Reglas Codex.
 4. Confirmar repositorio GitHub.
 5. Confirmar acceso a Supabase.
 6. Confirmar acceso a Vercel.
 7. Confirmar acceso a Telegram Bot si hiciera falta.
 8. Confirmar variables sin imprimir valores.
 9. Ejecutar prueba real de punta a punta.
-

29. Frase guía del proyecto

ParacasYa Market es el kiosco online de Paracas: una tienda local, rápida y confiable para pedir bebidas, snacks, hielo, cervezas, combos y productos básicos sin salir del hotel, Airbnb o casa.

30. Regla final

Si una tarea pone en riesgo pedidos, pagos, caja, Supabase, secrets, RLS o Edge Functions, detenerse y pedir confirmación.

La prioridad es:

No romper lo que ya funciona.